

CYB-050 : Formation à l'analyse forensique de base

Introduction Générale du Cours

Lorsqu'un incident de sécurité survient, la question immédiate n'est pas seulement de le contenir — c'est aussi de comprendre précisément ce qui s'est passé. Comment l'attaquant est-il entré ? Quels systèmes a-t-il compromis ? Quelles données a-t-il consultées, modifiées ou exfiltrées ? Depuis combien de temps était-il présent dans le réseau avant d'être détecté ? Ces questions ne peuvent recevoir de réponses fiables que si des preuves numériques ont été correctement collectées et analysées selon une méthodologie rigoureuse. C'est la mission de la forensique numérique.

La forensique numérique — également appelée informatique légale ou digital forensics — est la discipline scientifique qui consiste à identifier, préserver, analyser et présenter des preuves numériques de manière à garantir leur intégrité, leur authenticité et leur recevabilité. Elle s'applique aussi bien dans un contexte judiciaire (procédure pénale pour cybercriminalité) que dans un contexte de réponse à incident purement interne (investigation d'une compromission pour comprendre et remédier). Dans les deux cas, la rigueur méthodologique est non négociable : une preuve mal collectée est une preuve inutilisable.

Ce cours propose une initiation complète aux fondements et aux techniques de base de l'analyse forensique numérique. Nous aborderons les principes fondamentaux qui régissent toute investigation (ordre de volatilité, intégrité des preuves, chaîne de custody), les méthodes d'acquisition d'images disque et de dumps mémoire, l'analyse des artefacts Windows et Linux les plus révélateurs, la construction d'une timeline unifiée permettant de reconstituer la chronologie d'un incident, et l'utilisation des outils open source de référence (Autopsy, FTK Imager, Volatility, KAPE, Hayabusa, Plaso). L'objectif est de vous rendre capable de conduire une investigation forensique de premier niveau sur un système compromis et d'en produire un rapport structuré exploitable par les équipes de réponse à incident.

Objectifs Pédagogiques du Module

À l'issue de ce cours, l'apprenant doit être capable de :

1. Maîtrise Conceptuelle et Méthodologique

- Comprendre les principes fondamentaux de la forensique numérique : ordre de volatilité, intégrité des preuves, chaîne de custody, non-altération de l'original.
- Distinguer les contextes d'application de la forensique (judiciaire vs réponse à incident) et adapter la rigueur procédurale en conséquence.
- Appliquer une méthodologie d'investigation structurée en phases : identification, préservation, collecte, analyse, documentation, présentation.

2. Compétences Techniques d'Acquisition

- Réaliser une acquisition d'image disque bit-à-bit avec FTK Imager ou dd, avec vérification de l'intégrité par hash SHA-256.
- Effectuer un dump de la mémoire vive avec WinPmem (Windows) ou LiME (Linux) dans le respect de l'ordre de volatilité.
- Utiliser KAPE pour une collecte ciblée et rapide d'artefacts forensiques Windows en situation de triage.
- Monter et explorer une image forensique dans Autopsy sans modifier l'original.

3. Expertise en Analyse des Artefacts

- Identifier et analyser les artefacts forensiques Windows clés : journaux EVTX, registre, prefetch, MFT, LNK files, Amcache.
- Analyser les artefacts forensiques Linux : auth.log, bash_history, crontab, /etc/passwd, logs applicatifs.
- Réaliser une analyse de dump mémoire avec Volatility 3 pour identifier processus suspects, connexions et malwares fileless.
- Appliquer des règles YARA sur des images disque et des dumps mémoire pour détecter des malwares connus.

4. Construction de la Timeline et Documentation

- Construire une super-timeline unifiée à partir de sources multiples avec Plaso (log2timeline) et l'analyser avec Timesketch.
- Rédiger un rapport d'investigation forensique structuré, factuel et exploitable, adapté aux audiences techniques et légales.
- Documenter la chaîne de custody de manière rigoureuse pour garantir la recevabilité des preuves collectées.

Chapitre 1 : Principes Fondamentaux de la Forensique Numérique

1. Définition et Domaines d'Application

La forensique numérique est la discipline qui applique des méthodes scientifiques à la collecte, à la préservation, à l'analyse et à la présentation de preuves numériques. Son champ d'application couvre la forensique sur disque (analyse du contenu des supports de stockage), la forensique mémoire (analyse du contenu de la RAM), la forensique réseau (analyse des captures de trafic), la forensique mobile (analyse des smartphones et tablettes), et la forensique dans le cloud (analyse des logs et données des environnements cloud). Ce cours se concentre sur les trois premiers domaines, qui constituent le socle de toute investigation sur systèmes de bureau et serveurs.

La forensique numérique s'exerce dans deux contextes très différents qui impliquent des niveaux de rigueur procédurale distincts. Dans un contexte judiciaire — enquête pénale pour cybercriminalité, procédure civile, investigation disciplinaire — les preuves numériques peuvent être soumises à un tribunal et doivent satisfaire des critères stricts de recevabilité : intégrité vérifiable (hash calculé à la collecte et conservé séparément), traçabilité complète

(chaîne de custody documentée), méthodes reconnues (outils certifiés ou largement acceptés dans la communauté). Dans un contexte de réponse à incident interne — l'organisation cherche à comprendre une compromission pour y remédier et éviter la récurrence — les exigences procédurales sont moins contraignantes mais la rigueur méthodologique reste indispensable pour garantir la qualité des conclusions.

2. Le Principe de Non-Altération de la Preuve

Le principe fondateur de toute investigation forensique est la non-altération de l'original : aucune analyse ne doit être réalisée directement sur la preuve originale. Cette règle absolue protège l'intégrité de la preuve contre toute modification involontaire pouvant résulter de l'analyse elle-même. En forensique disque, cela se traduit par la création systématique d'une image bit-à-bit du support original, sur laquelle toutes les analyses sont réalisées — le support original est scellé et mis en sécurité dès la fin de l'acquisition. En forensique mémoire, le dump est réalisé une fois et toutes les analyses s'effectuent sur ce fichier de dump.

Le write blocker est l'équipement qui garantit physiquement la non-altération lors de l'acquisition. C'est un dispositif interposé entre le support original et le système d'acquisition, rendant toute écriture vers le support original physiquement impossible — même si le système d'acquisition tente d'écrire (par exemple, lors de la mise à jour des horodatages d'accès aux fichiers par le système d'exploitation). Des write blockers matériels (Tableau, WiebeTech) sont utilisés pour les disques durs physiques, tandis que des solutions logicielles (commande dd avec options appropriées, FTK Imager en mode lecture seule) sont utilisées pour les acquisitions distantes ou les machines virtuelles.

3. L'Ordre de Volatilité — Principe de Priorisation

L'ordre de volatilité est le principe forensique qui impose de collecter les données dans l'ordre décroissant de leur durée de vie, en commençant par les données les plus éphémères. Ce principe, formalisé dans le RFC 3227 (Guidelines for Evidence Collection and Archiving), est critique car certaines données disparaissent irrémédiablement à l'extinction du système — notamment le contenu de la RAM et les connexions réseau actives. Le tableau suivant présente l'ordre de volatilité standard avec les méthodes de collecte associées :

Priorité	Type de donnée	Durée de vie typique	Méthode de collecte
1	Registres CPU et cache	Quelques nanosecondes	Dump matériel spécialisé (rarement réalisable en pratique)
2	Mémoire vive (RAM)	Jusqu'à l'extinction du système	WinPmem (Windows), LiME (Linux), dump via hyperviseur (VM)
3	État des processus et connexions réseau	Jusqu'à l'extinction du système	Commandes live : ps, netstat, ss, tasklist, netstat -an
4	Fichiers temporaires et swap	Jusqu'à réécriture ou extinction	Inclusion dans l'image disque ou collecte ciblée /tmp, pagefile.sys

5	Journaux système (non centralisés)	Jusqu'à rotation des logs ou réécriture	Copie des fichiers de log avant toute action sur le système
6	Données sur disque (fichiers, MFT, registre)	Persistantes jusqu'à écrasement	Image disque bit-à-bit avec FTK Imager, dd, dcfldd
7	Données archivées et sauvegardes	Persistantes	Accès aux systèmes de backup, restauration sur environnement isolé

L'application de ce principe impose une décision stratégique dès les premières minutes d'une investigation sur système actif : faut-il éteindre le système (pour préserver l'état du disque et éviter l'effacement de traces par l'attaquant encore présent) ou le laisser allumé (pour collecter la RAM et les connexions réseau actives) ? Cette décision dépend du contexte : si le système héberge un malware actif communiquant avec un C2, le maintenir allumé permet de collecter l'adresse IP du C2 et les processus malveillants en mémoire — des informations souvent introuvables après extinction. Si le risque de persistance ou d'effacement de traces est élevé, l'isolation réseau immédiate suivie d'un dump mémoire et d'une extinction contrôlée est la procédure recommandée.

4. La Chaîne de Custody

La chaîne de custody (chain of custody) est la documentation continue et rigoureuse qui trace l'ensemble du cycle de vie d'une preuve numérique : depuis sa collecte initiale jusqu'à sa présentation en analyse, en passant par toutes les étapes de transfert, de stockage et de manipulation. Chaque action réalisée sur une preuve doit être documentée : qui l'a collectée, quand, depuis quel système, avec quel outil, quel hash a été calculé à la collecte, où la preuve est stockée, qui y a accédé ensuite et dans quel but.

En pratique, la chaîne de custody se matérialise sous forme d'un formulaire papier accompagnant physiquement les supports de stockage contenant les preuves, et d'un journal numérique documentant les opérations d'analyse. Chaque copie d'une image forensique génère un nouveau hash qui doit être vérifié et documenté. Un hash incorrect signale une altération potentielle de la copie, qui ne peut plus être utilisée comme preuve valide. Même dans un contexte de réponse à incident interne où la recevabilité judiciaire n'est pas l'enjeu immédiat, le maintien d'une chaîne de custody rigoureuse est une bonne pratique : si l'incident révèle ultérieurement une fraude interne ou une attaque criminelle, les preuves correctement documentées pourront être utilisées dans une procédure judiciaire.

Chapitre 2 : Acquisition des Preuves Numériques

1. Acquisition d'Image Disque

L'acquisition d'image disque est l'opération qui produit une copie bit-à-bit exacte d'un support de stockage. À la différence d'une simple copie de fichiers, l'image forensique inclut les espaces libres (qui peuvent contenir des fichiers supprimés récupérables), les métadonnées du système de fichiers, les secteurs cachés, la table de partition et tous les

artefacts du système de fichiers. Elle constitue la base sur laquelle toutes les analyses ultérieures sont réalisées, sans jamais toucher au support original.

FTK Imager (AccessData) est l'outil d'acquisition de référence sur Windows, disponible gratuitement. Son interface graphique guide l'investigateur à travers le processus : sélection de la source (disque physique, partition logique, image existante), choix du format de l'image (E01 — Expert Witness Format, avec compression et métadonnées intégrées, ou DD/RAW pour une compatibilité maximale), sélection de la destination, et vérification de l'intégrité par calcul automatique des hash MD5 et SHA-1 à la fin de l'acquisition. Sur Linux ou macOS, la commande `dd` combinée avec des outils de hash produit une image RAW : `dcfldd if=/dev/sdb of=/mnt/forensic/image.dd hash=sha256 bs=512 conv=noerror,sync` réalise l'acquisition avec calcul de hash simultané.

2. Formats d'Image Forensique

Plusieurs formats d'image forensique coexistent, chacun avec ses avantages spécifiques. Le format RAW (ou DD) est le plus simple : il produit une copie octet par octet du support sans aucune métadonnée additionnelle. Sa compatibilité est maximale — tous les outils forensiques peuvent lire ce format — mais il ne supporte pas nativement la compression ni les métadonnées (informations sur l'acquisition, hash, notes). Pour les disques de grande capacité, il peut générer des fichiers de plusieurs téraoctets difficiles à gérer.

Le format E01 (Expert Witness Format), développé par EnCase et devenu un standard de facto, supporte la compression, le découpage en segments de taille configurable (facilitant le stockage et le transfert) et l'intégration de métadonnées (nom de l'investigateur, numéro de cas, date d'acquisition, hash). Il est supporté par la quasi-totalité des outils forensiques majeurs (Autopsy, FTK, X-Ways). Le format AFF4 (Advanced Forensic File Format 4), plus récent et open source, offre des fonctionnalités avancées (chiffrement natif, compression performante, support des images distantes) et est notamment utilisé par Rekall et les outils forensiques modernes. Pour une première investigation, le format E01 ou DD représentent les choix les plus pragmatiques.

3. Acquisition de la Mémoire Vive

L'acquisition de la mémoire vive est une opération plus délicate que l'acquisition disque, car elle nécessite l'exécution d'un outil sur le système à analyser — ce qui modifie inévitablement légèrement le contenu de la mémoire. Cette modification est inévitable et acceptable : elle est documentée et ses effets sur l'investigation sont négligeables par rapport à la valeur des informations extraites. La règle est d'utiliser des outils dédiés, de taille minimale, qui interagissent le moins possible avec le système.

Sur Windows, WinPmem est l'outil de référence open source pour l'acquisition mémoire. Son exécution est simple : `winpmem_mini_x64.exe memory.dmp` produit un fichier de dump de la RAM complète dans le répertoire courant. L'outil supporte les systèmes Windows 7 à Windows 11 et produit un fichier au format AFF4 ou RAW selon les options choisies. Sur Linux, LiME (Linux Memory Extractor) est le module noyau de référence. Il s'insère dans le noyau en tant que module (`insmod lime.ko path=/mnt/forensic/memory.dmp format=lime`) et produit un dump de la mémoire physique complète. Il est indispensable de disposer du module compilé pour la version exacte du noyau du système à analyser — une préparation en amont est donc nécessaire pour les systèmes susceptibles d'être investigués.

4. Triage Forensique Rapide avec KAPE

Dans certains contextes opérationnels, l'acquisition d'une image disque complète n'est pas réalisable dans les délais imposés par l'incident — un serveur de production ne peut pas être mis hors ligne pendant les 4 à 8 heures nécessaires à l'acquisition d'un disque de 4 To. KAPE (Kroll Artifact Parser and Extractor) est conçu pour ces situations : il collecte uniquement les artefacts forensiques les plus pertinents (journaux, registre, prefetch, fichiers MFT, historique des navigateurs) sans acquérir l'intégralité du disque, en quelques minutes seulement.

KAPE fonctionne selon un système de Targets (définissant quels artefacts collecter) et de Modules (définissant quels outils appliquer aux artefacts collectés). Des targets préconfigurées couvrent les principales catégories d'artefacts Windows : les Event Logs, le Registre, les fichiers prefetch, la MFT, les artefacts de navigateurs (Chrome, Firefox, Edge), les journaux antivirus. Une collecte KAPE typique pour un triage de sécurité Windows s'exécute avec la commande `kape.exe --tsource C: --tdest D:\kape_output --target EventLogs,RegistryHives,Prefetch,MFT --msource D:\kape_output --mdest D:\kape_output\processed --module EvtxECmd,RECmd,PECmd`. En 5 à 15 minutes, l'investigateur dispose d'un ensemble d'artefacts parsés et structurés sur lequel débiter l'analyse.

Chapitre 3 : Outils Open Source de Forensique

1. Panorama des Outils

L'écosystème des outils open source de forensique numérique est riche et couvre l'ensemble des phases d'une investigation. Le tableau suivant présente les principaux outils, leurs licences et leurs cas d'usage dans une investigation :

Outil	Licence	Fonction principale	Cas d'usage typique
Autopsy	Open source (gratuit)	Suite forensique complète — analyse d'images disque, timeline, récupération de fichiers supprimés, recherche de mots-clés	Investigation principale sur image disque Windows ou Linux
FTK Imager	Gratuit (AccessData)	Acquisition d'image disque bit-à-bit, montage d'images forensiques, calcul de hash, aperçu de l'arborescence	Acquisition sur site, vérification d'intégrité des images

Volatility 3	Open source (gratuit)	Analyse de dumps mémoire RAM — processus, connexions, injections, malwares fileless, extraction de credentials	Investigation mémoire sur systèmes Windows, Linux, macOS
KAPE (Kroll Artifact Parser)	Gratuit (Kroll)	Collecte ciblée et rapide d'artefacts forensiques Windows — journaux, registre, prefetch, navigateurs, sans image disque complète	Triage rapide sur site quand l'image complète n'est pas possible
Hayabusa	Open source (gratuit)	Analyse rapide des journaux Windows EVTX — génération de timeline, détection basée sur règles Sigma, rapport CSV/HTML	Première analyse des journaux Windows d'un système suspect
Plaso / log2timeline	Open source (gratuit)	Génération de super-timeline à partir de multiples sources (logs, artefacts disque, registre, navigateurs)	Construction de la chronologie complète d'un incident
YARA	Open source (gratuit)	Détection de malwares par patterns — règles de correspondance sur fichiers, processus en mémoire ou images disque	Recherche de malwares connus ou de familles de malwares dans une image
Wireshark / tshark	Open source (gratuit)	Analyse de captures réseau (PCAP) — reconstruction des flux, détection de communications C2, extraction de fichiers transférés	Analyse des communications réseau pendant l'incident

2. Autopsy — Suite Forensique Open Source

Autopsy est la suite forensique open source la plus complète disponible gratuitement. Développée par Basis Technology, elle offre une interface graphique permettant d'analyser des images disque Windows, Linux et Android sans modifier les données originales. Son interface s'organise autour de cas (chaque investigation constitue un cas distinct avec ses sources, ses résultats et ses rapports) et de modules d'ingestion (Ingest Modules) qui analysent automatiquement l'image lors de son ouverture.

Les modules d'ingestion d'Autopsy couvrent de nombreux domaines analytiques : la récupération de fichiers supprimés (Carved Files), l'analyse des métadonnées EXIF des images, la génération de hashes pour comparaison avec des bases de données (NSRL — National Software Reference Library — pour identifier les fichiers système légitimes), l'analyse des artefacts de navigateurs (historique, téléchargements, cookies), l'extraction

des activités récentes (fichiers récents, programmes lancés), la génération d'une timeline des activités sur le système. La fonctionnalité de recherche par mots-clés (Keyword Search) permet de rechercher des chaînes de caractères ou des expressions régulières dans l'intégralité de l'image, y compris dans les espaces libres et les fichiers supprimés — une capacité essentielle pour identifier des traces d'exfiltration ou des documents sensibles accédés.

3. Hayabusa — Analyse Rapide des Journaux Windows

Hayabusa est un outil open source japonais conçu pour l'analyse rapide des journaux Windows au format EVTX. Sa conception orientée performance lui permet de traiter en quelques secondes des milliers de fichiers de journaux, en appliquant simultanément des centaines de règles de détection au format Sigma. Il produit une sortie chronologique des événements significatifs, avec pour chaque événement le niveau d'alerte (critique, élevé, modéré), la règle Sigma déclenchée, le mapping MITRE ATT&CK et les champs pertinents de l'événement.

L'utilisation de Hayabusa est particulièrement adaptée à la phase initiale d'une investigation Windows, avant de plonger dans l'analyse détaillée avec Autopsy : la commande `hayabusa.exe csv-timeline -d chemin\vers\evtx -o timeline.csv --no-summary` produit en quelques secondes un fichier CSV chronologique de tous les événements significatifs, triable et filtrable dans Excel ou un SIEM. Les règles Sigma intégrées couvrent plusieurs centaines de scénarios d'attaque : brute force, création de comptes, exécution de PowerShell malveillant, création de services, modification du pare-feu Windows, utilisation de wevtutil pour effacer les journaux. Ce premier balayage oriente immédiatement l'investigation vers les événements les plus significatifs.

4. YARA — Détection par Patterns

YARA est un langage de description de patterns permettant de créer des règles de détection de malwares basées sur des caractéristiques de leur contenu — chaînes de caractères, séquences d'octets, expressions régulières. Une règle YARA peut décrire une famille de malwares par ses strings caractéristiques (messages d'erreur intégrés, noms d'API Windows spécifiques, chemins de fichiers), ses patterns binaires (séquences d'octets composant une fonction ou un shellcode reconnaissable), ou une combinaison des deux. Les règles YARA peuvent être appliquées sur des fichiers individuels, des répertoires complets, des images disque ou des dumps mémoire.

De nombreuses bibliothèques de règles YARA publiques couvrent les familles de malwares les plus répandues : les règles Mandiant (ex-FireEye), les règles Elastic, les règles de la communauté VirusTotal. La commande `yara -r regles.yar image.dd` applique récursivement l'ensemble des règles du fichier `regles.yar` sur l'image disque et signale chaque correspondance avec le nom de la règle déclenchée et le décalage dans l'image. En forensique mémoire, la commande `yara -r regles.yar memory.dmp` permet de détecter des malwares fileless dont aucune trace n'existe sur le disque. Cette capacité de détection cross-source — disque et mémoire — est l'un des atouts majeurs de YARA dans une investigation forensique.

Chapitre 4 : Analyse des Artefacts Windows

1. Panorama des Artefacts Windows

Windows est un système d'exploitation remarquablement riche en artefacts forensiques. Chaque action réalisée sur un système Windows — ouverture d'un fichier, exécution d'un programme, connexion réseau, installation d'un service, modification du registre — laisse des traces dans plusieurs artefacts distincts. Cette redondance est un atout majeur pour l'investigation : même si un attaquant efface certaines traces, d'autres subsistent dans des artefacts qu'il n'a pas pensé à nettoyer. Le tableau suivant présente les artefacts Windows les plus importants pour une investigation de sécurité :

Artefact	Localisation	Informations forensiques exploitables
Journaux d'événements (EVTX)	C:\Windows\System32\winevt\Logs\	Authentifications (4624/4625), création de processus (4688), services installés (7045), journal effacé (1102) — source principale de l'investigation Windows
Registre Windows	C:\Windows\System32\config\ (ruches système) + NTUSER.DAT (ruche utilisateur)	Programmes au démarrage (Run keys), services, fichiers récents ouverts (MRU), logiciels installés, profils réseau WiFi
Prefetch	C:\Windows\Prefetch\	Liste des exécutables lancés avec horodatage des 8 dernières exécutions — preuve d'exécution d'un programme même supprimé
MFT (Master File Table)	Racine du volume NTFS (fichier \$MFT)	Métadonnées de tous les fichiers du volume — création, modification, accès, suppression — y compris les fichiers effacés
LNK files (raccourcis)	C:\Users\%user%\AppData\Roaming\Microsoft\Windows\Recent\	Fichiers récemment ouverts avec chemin d'accès complet, horodatage, et informations sur le volume source (preuve d'utilisation de supports amovibles)
NTDS.dit	C:\Windows\NTDS\ (sur les contrôleurs de domaine)	Base de données Active Directory — hashes des mots de passe de tous les comptes du domaine (cible prioritaire des attaquants)
Historique navigateurs	AppData\Local\Google\Chrome\User Data\Default\ (et équivalents)	Historique de navigation, téléchargements, mots de passe enregistrés, cookies, cache — trace des activités web de l'utilisateur
Amcache.hve / ShimCache	C:\Windows\AppCompat\Programs\Amcache.hve	Liste des exécutables ayant tourné sur le système avec leur hash SHA-1 — complémentaire au Prefetch pour prouver l'exécution

2. Analyse des Journaux d'Événements Windows

Les journaux d'événements Windows (fichiers EVTX) sont la source d'information la plus riche pour reconstituer la chronologie d'un incident sur un système Windows. Ils sont stockés dans `C:\Windows\System32\winevt\Logs\` et organisés en plusieurs journaux thématiques. Le journal `Security.evtx` contient les événements d'authentification et de gestion des comptes — c'est le journal prioritaire dans toute investigation de sécurité. Le journal `System.evtx` contient les événements système (démarrage, arrêt, installation de services). Le journal `Application.evtx` contient les événements des applications. Des journaux spécialisés comme `Microsoft-Windows-PowerShell/Operational.evtx` (exécution de scripts PowerShell) et `Microsoft-Windows-Sysmon/Operational.evtx` (si Sysmon est déployé, avec une télémétrie très détaillée sur les processus, connexions et modifications de fichiers) sont particulièrement précieux.

L'Event ID 4688 (création d'un nouveau processus) est l'un des plus importants en investigation de sécurité : il enregistre le nom du processus créé, son PID, le processus parent, et (si la politique d'audit est correctement configurée) la ligne de commande complète. Une séquence `winword.exe` (ID 4688) → `cmd.exe` (4688) → `powershell.exe -EncodedCommand [...]` (4688) → connexion réseau sortante est le pattern caractéristique d'un document Word malveillant exécutant un payload PowerShell. L'Event ID 1102 (journal d'audit effacé) est un signal d'alerte critique signalant une tentative de dissimulation des traces — l'heure de cet événement délimite la fenêtre temporelle avant laquelle l'investigation devra se concentrer sur d'autres sources d'artefacts.

3. Analyse du Registre Windows

Le registre Windows est une base de données hiérarchique centralisant les paramètres de configuration du système et des applications. Pour l'investigation forensique, il constitue une source précieuse d'informations sur la persistance des malwares et les activités récentes. Les clés `Run` et `RunOnce` sous `HKLM\Software\Microsoft\Windows\CurrentVersion\Run` et `HKCU\Software\Microsoft\Windows\CurrentVersion\Run` listent les programmes démarrés automatiquement à chaque ouverture de session — un vecteur de persistance classique pour les malwares qui y inscrivent une clé pointant vers leur exécutable.

La ruche `NTUSER.DAT` (propre à chaque utilisateur) contient des artefacts d'activité utilisateur très riches. Les clés `MRU` (Most Recently Used) répertorient les derniers fichiers ouverts dans chaque application. La clé `UserAssist` enregistre (sous forme obfusquée en ROT13) tous les programmes lancés par l'utilisateur avec leur fréquence d'utilisation et leur dernier horodatage d'exécution. `ShellBags` enregistre les dossiers que l'utilisateur a navigués — y compris des dossiers réseau ou sur des supports amovibles qui n'existent plus. Ces artefacts permettent de reconstituer l'activité d'un utilisateur même si les journaux d'événements ont été effacés. L'outil `RegRipper` automatise l'extraction et la présentation de ces données depuis les fichiers de ruches extraits de l'image disque.

4. Analyse du Prefetch et de la MFT

Le prefetch est un mécanisme Windows qui précharge les ressources utilisées par les applications pour accélérer leurs démarrages ultérieurs. Chaque application exécutée génère un fichier `.pf` dans `C:\Windows\Prefetch\`, nommé d'après l'exécutable et un hash de son chemin. Ces fichiers conservent les 8 derniers horodatages d'exécution de l'application, la liste des fichiers et DLL qu'elle a chargés, et le nombre total d'exécutions. Forensiquement, le prefetch constitue une preuve d'exécution : même si l'exécutable

malveillant a été supprimé, son fichier prefetch peut subsister et attester de son exécution, de sa localisation au moment de l'exécution et des ressources qu'il utilisait. L'outil PECmd (Eric Zimmerman Tools) parse les fichiers prefetch et produit une sortie lisible incluant tous ces artefacts.

La Master File Table (MFT) est la structure centrale du système de fichiers NTFS répertoriant tous les fichiers et répertoires du volume avec leurs métadonnées complètes. Chaque entrée MFT contient quatre horodatages pour chaque fichier : la date de création, la date de dernière modification du contenu, la date de dernière modification des métadonnées (\$MFT Change), et la date de dernier accès (\$STDINFO). L'analyse forensique de la MFT permet de détecter les manipulations de timestamps (timestamping) — une technique utilisée par les attaquants pour modifier les horodatages des fichiers malveillants et les faire passer pour des fichiers système légitimes. Une discordance entre les horodatages \$STDINFO et \$FILE_NAME d'un même fichier est un signal fort de manipulation. MFTECmd (Eric Zimmerman Tools) parse la MFT et produit une timeline complète de tous les fichiers du volume.

Chapitre 5 : Analyse des Artefacts Linux

1. Panorama des Artefacts Linux

Les systèmes Linux présentent une philosophie de journalisation différente de Windows : les logs sont des fichiers texte stockés dans /var/log/ et lisibles directement, sans outil spécifique. Cette simplicité est un avantage pour l'investigation — les logs sont directement grep-ables et analysables avec les outils système standard — mais présente aussi un risque : un attaquant ayant obtenu les droits root peut modifier ou supprimer ces fichiers sans laisser de trace dans un journal tamper-proof (contrairement aux journaux Windows qui nécessitent des droits spécifiques pour être effacés). Le tableau suivant présente les artefacts Linux essentiels pour une investigation de sécurité :

Artefact	Chemin	Informations forensiques exploitables
auth.log / secure	/var/log/auth.log (Debian/Ubuntu) ou /var/log/secure (RHEL/CentOS)	Authentifications SSH (réussies et échouées), utilisation de sudo, changements de compte — source principale de l'investigation Linux
syslog / messages	/var/log/syslog ou /var/log/messages	Événements système généraux, démarrage et arrêt de services, messages du noyau
Bash history	/home/%user%/.bash_history et /root/.bash_history	Historique des commandes exécutées — attention : facilement falsifiable ou effacé par l'attaquant
Crontab utilisateur et système	/var/spool/cron/crontabs/ et /etc/cron.*	Tâches planifiées — vecteur de persistance classique (backdoor relancée périodiquement)

<code>/etc/passwd</code> et <code>/etc/shadow</code>	<code>/etc/passwd</code> , <code>/etc/shadow</code>	Liste des comptes utilisateurs — présence de comptes inconnus avec UID 0 (root) ou shell actif
Fichiers SUID/SGID suspects	Recherche via <code>find / -perm -4000</code> ou <code>-2000</code>	Binaires avec bit SUID permettant l'élévation de privilèges — vecteur d'exploitation courant
Connexions réseau actives	Sortie de <code>ss -tlnp</code> ou <code>netstat -tlnp</code> au moment de l'incident	Services en écoute, connexions établies vers l'extérieur — identification du C2 potentiel
Logs Apache / Nginx	<code>/var/log/apache2/</code> ou <code>/var/log/nginx/</code>	Requêtes HTTP reçues — identification d'un webshell utilisé, requêtes d'exploitation, exfiltration via HTTP

2. Analyse des Journaux d'Authentification

Le fichier `/var/log/auth.log` (Debian/Ubuntu) ou `/var/log/secure` (RHEL/CentOS) est le journal d'authentification de référence sur Linux. Il enregistre toutes les tentatives de connexion SSH (réussies et échouées avec l'adresse IP source), les élévations de privilèges via `sudo` (avec la commande exécutée), les changements de compte (`su`), et les authentifications PAM. L'analyse de ce fichier pour une investigation commence typiquement par la recherche des connexions réussies depuis des adresses IP externes (`grep "Accepted" /var/log/auth.log`) pour identifier le patient zéro ou les machines de rebond utilisées par l'attaquant.

Le journal `/var/log/wtmp` (accès via la commande `last`) enregistre toutes les sessions de connexion et déconnexion avec l'horodatage et l'adresse IP source. La commande `lastb` (journal `/var/log/btmp`) liste tous les échecs d'authentification. Ces journaux binaires sont plus difficiles à falsifier que les fichiers texte et constituent donc des sources forensiques plus fiables. Sur les systèmes utilisant `systemd`, `journalctl` offre un accès structuré à l'ensemble des journaux système avec des options de filtrage puissantes : `journalctl -u ssh --since "2024-11-01" --until "2024-11-15"` extrait tous les événements SSH sur une période définie.

3. Analyse de la Persistance sous Linux

La persistance est le mécanisme par lequel un attaquant garantit que son accès au système sera maintenu après un redémarrage ou une déconnexion. Sur Linux, plusieurs mécanismes sont couramment utilisés. Les crontabs — tables de planification de tâches — peuvent contenir des commandes malveillantes exécutées périodiquement : un backdoor qui se reconnecte au C2 toutes les heures, ou un script qui maintient actif un reverse shell. L'investigation vérifie systématiquement les crontabs de tous les utilisateurs (`/var/spool/cron/crontabs/`) et les crontabs système (`/etc/crontab`, `/etc/cron.d/`).

Les services `systemd` constituent un autre vecteur de persistance : un attaquant ayant les droits root peut créer un service malveillant dans `/etc/systemd/system/` ou `/usr/lib/systemd/system/` qui sera démarré automatiquement au boot. La commande `systemctl list-units --type=service` liste tous les services actifs, et l'analyse des fichiers `.service` récemment modifiés (`find /etc/systemd/system/ -newer /var/log/auth.log -type f`) peut révéler des services ajoutés après le début de l'incident. Les fichiers `.bashrc`, `.bash_profile` et `.profile` des utilisateurs sont également des vecteurs de persistance simples : l'ajout d'une

commande à la fin de ces fichiers la fait exécuter à chaque ouverture de session de l'utilisateur.

4. Analyse des Serveurs Web Compromis

Les serveurs web exposés sur Internet sont des cibles fréquentes. Leur compromission laisse des traces spécifiques dans les logs d'accès Apache ou Nginx. L'analyse forensique de ces logs recherche plusieurs patterns caractéristiques. Les tentatives d'exploitation web (injection SQL, path traversal, upload de fichiers malveillants) se manifestent par des requêtes contenant des patterns suspects : `union+select`, `../../../../`, `cmd.exe`, `/etc/passwd` dans l'URI. Un pic soudain de codes de retour 500 (erreur serveur interne) peut indiquer des tentatives d'exploitation réussies ou échouées.

La présence d'un webshell — un script PHP, Python ou Perl déposé sur le serveur permettant l'exécution de commandes distantes via HTTP — se détecte dans les logs par des requêtes POST vers un fichier à l'emplacement inhabituel avec des paramètres suspects. La recherche de webshells dans le répertoire web consiste à identifier les fonctions d'exécution de commandes caractéristiques (`system`, `exec`, `passthru`, `shell_exec`, `eval`) dans les fichiers PHP. Des outils dédiés comme `php-malware-finder` ou `maldet` automatisent cette recherche avec des bases de signatures de webshells connus.

Chapitre 6 : Analyse de la Mémoire avec Volatility

1. Pourquoi Analyser la Mémoire Vive

La mémoire vive (RAM) est le terrain de jeu des malwares modernes les plus sophistiqués. Les malwares fileless — qui s'exécutent exclusivement en mémoire sans jamais écrire de fichier persistant sur le disque — contournent les antivirus basés sur la détection de fichiers et ne laissent aucune trace sur le disque après extinction du système. Ils ne sont visibles que dans la mémoire d'un système en cours d'exécution. Même pour les malwares conventionnels qui écrivent des fichiers, l'analyse mémoire révèle des informations inaccessibles depuis le disque : les connexions réseau actives au moment de l'incident, les clés de chiffrement utilisées par un ransomware, les injections de code dans des processus légitimes, et les credentials récupérés par un outil de type Mimikatz depuis le processus LSASS.

Volatility est le framework open source de référence pour l'analyse de dumps mémoire. Sa version 3 (Volatility Foundation) supporte les dumps mémoire Windows (toutes versions depuis XP), Linux (avec les profils adaptés au noyau) et macOS. Il propose une architecture de plugins permettant d'analyser chaque aspect de la mémoire : processus, connexions réseau, injections de code, fichiers en mémoire, registre en cache, et bien d'autres. L'ensemble des analyses Volatility sont réalisées sur le fichier de dump — le système source n'est plus impliqué une fois le dump réalisé.

2. Principaux Plugins Volatility 3

Volatility 3 est utilisé en ligne de commande : `python3 vol.py -f memory.dmp [plugin]`. Le tableau suivant présente les plugins les plus utilisés dans une investigation de sécurité, avec les commandes associées et les informations extraites :

Plugin Volatility 3	Commande	Informations extraites
windows.pslist	<code>vol.py -f memory.dmp windows.pslist</code>	Liste des processus actifs avec PID, PPID, date de création — vue globale de l'activité au moment du dump
windows.pstree	<code>vol.py -f memory.dmp windows.pstree</code>	Arborescence parent-enfant des processus — détection des processus légitimes avec des enfants suspects (ex : Word lançant PowerShell)
windows.netscan	<code>vol.py -f memory.dmp windows.netscan</code>	Connexions réseau actives et fermées au moment du dump — identification des communications C2 actives
windows.malfind	<code>vol.py -f memory.dmp windows.malfind</code>	Régions mémoire suspectes (exécutables injectés) — détection de l'injection de code et des malwares fileless
windows.dlllist	<code>vol.py -f memory.dmp windows.dlllist --pid <PID></code>	DLL chargées par un processus — détection des DLL malveillantes ou des DLL hijacking
windows.hashdump	<code>vol.py -f memory.dmp windows.hashdump</code>	Hashes NTLM des comptes locaux depuis la mémoire du processus LSASS — récupération de credentials
windows.cmdline	<code>vol.py -f memory.dmp windows.cmdline</code>	Lignes de commande complètes de chaque processus — reconstitution des commandes exécutées par l'attaquant

3. Détection d'Injection de Code

L'injection de code est une technique par laquelle un malware insère son code malveillant dans l'espace mémoire d'un processus légitime (`svchost.exe`, `explorer.exe`, `lsass.exe`) pour s'exécuter sous couverture d'un processus de confiance et contourner les contrôles de sécurité basés sur la liste blanche des processus. Le plugin `windows.malfind` de Volatility est conçu pour détecter ces injections : il recherche les régions mémoire marquées à la fois comme exécutables et accessibles en lecture-écriture (combinaison caractéristique du code injecté), contenant une entête PE (MZ header) — signature d'un exécutable Windows.

L'analyse des résultats de `malfind` nécessite un tri : de nombreuses régions signalées sont des faux positifs (code légitime utilisant cette configuration de permissions pour des raisons techniques). Les indicateurs d'une véritable injection sont : la présence d'un MZ header en début de région signalée, un processus hôte inhabituel (`svchost` avec un parent autre que `services.exe`, `explorer` avec des connexions réseau sortantes inhabituelle), et la

correspondance du code injecté avec des signatures de malwares connus lors de l'application de règles YARA sur la région extraite. La commande `vol.py -f memory.dmp windows.dumpfiles --pid <PID>` extrait le code injecté dans un fichier pour analyse statique avec un outil de désassemblage ou vérification sur VirusTotal.

4. Extraction des Credentials depuis la Mémoire

Mimikatz est l'outil le plus utilisé par les attaquants pour extraire des credentials depuis la mémoire Windows. Il cible principalement le processus LSASS (Local Security Authority Subsystem Service) qui maintient en mémoire les credentials des sessions actives sous forme de hashes NTLM ou, sur les configurations vulnérables (WDigest activé), en clair. Le plugin `windows.hashdump` de Volatility permet de réaliser la même extraction depuis un dump mémoire, sans exécuter Mimikatz sur le système cible. Les hashes ainsi extraits peuvent être utilisés pour comprendre l'étendue de la compromission (quels comptes étaient connectés au moment de l'incident ?) et pour vérifier si un attaquant aurait pu récupérer des credentials de comptes à privilèges.

La détection de l'utilisation de Mimikatz dans un dump mémoire s'effectue via les strings caractéristiques que l'outil laisse en mémoire lors de son exécution (`sekurlsa`, `lsadump`, `kerberos :: list`, `mimikatz`) ou via des règles YARA spécifiques à Mimikatz disponibles dans les bibliothèques publiques. Ces détections dans le dump mémoire constituent des preuves de credential dumping même si aucune trace de Mimikatz n'est retrouvée sur le disque — confirmant l'utilisation d'une technique fileless ou d'un effacement des traces par l'attaquant.

Chapitre 7 : Construction de la Timeline et Rapport

1. La Super-Timeline — Outil Central de Reconstitution

La super-timeline est une chronologie unifiée agrégeant des événements horodatés issus de toutes les sources forensiques disponibles : MFT (création, modification, accès aux fichiers), journaux d'événements Windows, journaux système Linux, artefacts de registre, prefetch, historique des navigateurs, logs réseau, et tout autre artefact horodaté. Elle constitue le document central de l'investigation forensique, permettant de reconstituer minute par minute la séquence des événements survenus pendant l'incident — depuis l'accès initial jusqu'aux actions finales de l'attaquant.

Plaso (Python Log2timeline) est l'outil open source de référence pour la génération de super-timelines. Il supporte des dizaines de formats de sources (logs Windows EVTX, artefacts NTFS, journaux Linux, artefacts macOS, logs réseau, bases de données SQLite des navigateurs, exports SIEM) et produit un fichier de sortie au format Plaso (`.plaso`) contenant l'ensemble des événements normalisés. La commande `log2timeline.py --parsers win7 timeline.plaso image.dd` traite une image Windows et génère le fichier de timeline. `Psort.py` (inclus dans Plaso) filtre, trie et exporte les événements en CSV ou en JSON pour analyse dans un tableur, un SIEM ou Timesketch.

2. Analyse de la Timeline avec Timesketch

Timesketch est une plateforme web open source développée par Google pour la visualisation et l'analyse collaborative de super-timelines forensiques. Elle permet d'importer les timelines Plaso ou CSV, de les visualiser chronologiquement, d'effectuer des recherches full-text, d'annoter les événements significatifs, de partager les analyses entre plusieurs investigateurs, et de définir des stories — des narratifs documentant la reconstruction de l'incident. L'interface graphique permet de filtrer la timeline sur des fenêtres temporelles spécifiques, de rechercher des termes clés (un nom de processus, une adresse IP, un nom d'utilisateur) et de visualiser visuellement les pics d'activité.

L'analyse d'une super-timeline commence par l'identification des événements de haute importance qui serviront d'ancres chronologiques : la première connexion SSH depuis une IP externe inconnue, la première exécution d'un outil malveillant (détecté par une règle Hayabusa ou YARA), la création d'un fichier suspect, le moment de l'alerte SIEM. Autour de chaque ancre, l'analyste explore la timeline pour comprendre ce qui s'est passé avant (comment l'attaquant est arrivé à ce point) et après (quelles actions a-t-il enchaînées). Cette exploration chronologique est itérative et guide progressivement vers une compréhension complète de l'incident.

3. Structure du Rapport d'Investigation Forensique

Le rapport d'investigation forensique est le document qui traduit les résultats techniques de l'analyse en conclusions exploitables par les décideurs et les équipes opérationnelles. Sa qualité détermine la valeur de l'investigation : des conclusions techniques impeccables mal documentées perdent leur impact opérationnel et leur recevabilité légale potentielle. Un rapport forensique complet comporte plusieurs sections distinctes, chacune adressant une audience ou un besoin spécifique.

La synthèse exécutive (une à deux pages) est lisible par la direction sans connaissances techniques : elle décrit l'incident en termes de chronologie, d'impact (systèmes compromis, données potentiellement exposées), de vecteur d'attaque et de recommandations prioritaires. Le résumé technique (cinq à dix pages) détaille la méthodologie d'investigation, les sources analysées, la chaîne de custody, et les conclusions principales avec leurs preuves. L'annexe technique contient les données brutes : extraits des artefacts pertinents (captures d'écran Autopsy, extraits de logs, sorties Volatility), la liste des IOC identifiés, les hashes des fichiers malveillants trouvés, et les logs de toutes les commandes exécutées durant l'investigation (traçabilité complète). Cette dernière section garantit que les conclusions sont vérifiables par un tiers expert.

4. Bonnes Pratiques de Documentation

La documentation est une discipline à part entière dans l'investigation forensique. Elle doit démarrer dès les premières secondes de l'investigation et se poursuivre jusqu'à la remise du rapport final. Chaque action réalisée doit être documentée en temps réel — l'investigateur ne doit jamais compter sur sa mémoire pour reconstituer a posteriori la séquence de ses actions. La prise de notes en temps réel peut s'effectuer sur un carnet papier (inaltérable numériquement), dans un outil de prise de notes (CherryTree, Obsidian) ou dans un outil de case management (TheHive).

Pour chaque outil exécuté, la documentation inclut : la commande exacte exécutée avec tous ses paramètres, l'horodatage d'exécution, la source traitée et sa hash, le fichier de sortie produit et sa hash. Cette rigueur garantit que l'investigation est reproductible : un deuxième investigateur disposant des mêmes sources et de la documentation complète doit pouvoir reproduire les mêmes analyses et obtenir les mêmes résultats. La reproductibilité est un critère de recevabilité essentiel dans un contexte judiciaire et un gage de qualité scientifique dans tout contexte.

Chapitre 8 : Cas Pratique — Investigation d'un Serveur Compromis

1. Présentation du Scénario

Le lundi 18 novembre, 08h45, le SOC de TechEdge SA reçoit une alerte Wazuh critique : le module FIM signale la modification du fichier `/etc/passwd` sur le serveur Linux SRV-WEB-01, qui héberge le site corporate de l'entreprise (Apache + PHP + WordPress). Simultanément, une alerte pare-feu signale des connexions sortantes depuis SRV-WEB-01 vers une IP externe (91.220.131.45) sur le port TCP/4444 — un port typiquement associé aux reverse shells Metasploit. Le serveur est en production depuis 3 ans et n'a pas été patché depuis 6 mois selon la CMDB.

La décision de réponse initiale est d'isoler immédiatement SRV-WEB-01 du réseau via l'EDR (coupure des interfaces réseau tout en maintenant la machine accessible depuis le réseau de management pour l'investigation) avant de procéder à l'acquisition forensique. L'analyste N2 est mandaté pour conduire l'investigation et produire un rapport sous 48 heures.

2. Phase d'Acquisition

L'analyste se connecte au serveur via le réseau de management et commence par documenter l'état du système en temps réel (ordre de volatilité) : capture des connexions réseau actives (`ss -tlnp > /tmp/netstat.txt`), liste des processus actifs (`ps auxf > /tmp/ps.txt`), liste des fichiers ouverts (`lssof > /tmp/lssof.txt`). Ces fichiers sont immédiatement copiés vers un poste d'analyse via SCP.

Le dump mémoire est réalisé avec LiME : le module est compilé pour le noyau exact du serveur (`uname -r` révèle kernel 5.15.0-89-generic), chargé avec `insmod lime.ko path=/tmp/memory.dmp format=lime`, et le fichier de 8 Go produit est copié vers le poste d'analyse. Le hash SHA-256 est calculé immédiatement et consigné dans le journal d'investigation. L'image disque est ensuite réalisée avec FTK Imager depuis le réseau de management, produisant une image E01 de 120 Go du disque système. Le hash SHA-256 de l'image est vérifié par FTK Imager à la fin de l'acquisition. Le serveur reste isolé pendant toute la durée de l'investigation.

3. Phase d'Analyse

L'analyse des logs Apache (/var/log/apache2/access.log) révèle la séquence d'attaque initiale. Le 15 novembre à 23h44, une série de requêtes POST vers /wp-admin/theme-editor.php présente des paramètres caractéristiques d'une exploitation de vulnérabilité WordPress (CVE-2024-XXXX, injection de code dans l'éditeur de thème). À 23h47, une requête GET vers /wp-content/themes/twentytwentythree/shell.php?cmd=id retourne une réponse 200 (succès) — confirmation de la création et de l'utilisation d'un webshell PHP. L'analyse de /var/log/auth.log révèle une connexion SSH réussie depuis l'IP 91.220.131.45 le 16 novembre à 00h12, avec le compte www-data — compte de service Apache anormalement configuré avec un accès SSH actif.

L'analyse du dump mémoire avec Volatility révèle un processus python3 lancé par www-data avec la ligne de commande `python3 -c 'import socket,subprocess,os;s=socket.socket(...)` — un reverse shell Python classique. Ce processus est le processus parent d'un shell bash, depuis lequel les commandes d'élévation de privilèges ont été exécutées (exploitation de CVE-2023-2640, vulnérabilité OverlayFS permettant l'élévation vers root sur Ubuntu 22.04). La modification de /etc/passwd observée par le FIM correspond à l'ajout d'un compte backdoor `r00t:x:0:0::root:/bin/bash` avec UID 0 (droits root). La super-timeline Plaso confirme la séquence : exploitation web (23h44) → webshell déposé (23h47) → connexion SSH www-data (00h12) → élévation de privilèges (00h23) → création compte backdoor (00h31) → reverse shell persistant (00h35).

4. Conclusions et Recommandations

Le rapport d'investigation identifie trois causes racines. La première est l'absence de patch management : la vulnérabilité WordPress exploitée était publiée depuis 4 mois avec un correctif disponible depuis le lendemain de sa publication. La deuxième est la configuration incorrecte du compte www-data avec un accès SSH actif et un shell valide (/bin/bash au lieu de /usr/sbin/nologin), transformant un compte de service sans droits en vecteur d'accès interactif. La troisième est l'absence de WAF en frontal du serveur WordPress, qui aurait pu détecter et bloquer les requêtes d'exploitation.

Les IOC identifiés sont immédiatement partagés avec le SIEM pour alerte sur d'autres systèmes : l'IP 91.220.131.45, le hash SHA-256 du webshell (déposé dans /wp-content/themes/twentytwentythree/shell.php), le hash du module LKM utilisé pour l'élévation de privilèges, et le pattern de la ligne de commande du reverse shell Python. La recherche sur les autres serveurs via l'EDR ne révèle aucune compromission supplémentaire. Le RGPD impose de vérifier si des données personnelles ont été exposées : l'analyse des requêtes HTTP sortantes depuis le webshell ne révèle pas d'exfiltration de données, mais la CNIL est notifiée par précaution dans le délai de 72 heures imposé en cas de doute sur l'exposition de données personnelles.

Synthèse Générale du Module

Ce cours vous a conduit depuis les principes fondamentaux de la forensique numérique jusqu'à la conduite complète d'une investigation sur un serveur Linux compromis, en passant par les techniques d'acquisition, l'analyse des artefacts Windows et Linux, l'investigation mémoire avec Volatility et la construction de la super-timeline permettant de reconstituer la chronologie d'un incident.

Trois grandes leçons synthétisent l'ensemble du cours. La première est que la rigueur procédurale n'est pas une option. La forensique numérique tire sa valeur de la confiance que l'on peut accorder à ses conclusions : une preuve dont l'intégrité n'est pas garantie (pas de hash calculé à la collecte), une investigation dont les actions ne sont pas tracées (pas de documentation en temps réel), ou une analyse réalisée sur l'original plutôt que sur une copie ne produisent pas des conclusions fiables. Cette rigueur n'est pas une contrainte bureaucratique — c'est la condition sine qua non de la qualité scientifique de l'investigation.

La deuxième leçon est que l'ordre de volatilité est une discipline à intégrer automatiquement. Lors d'une intervention sur un système actif suspect, chaque seconde compte : la RAM contient des informations qui disparaîtront à l'extinction et qui ne seront jamais récupérables. Avant toute autre action — avant même de commencer à analyser les logs sur le système — l'analyste doit capturer la mémoire vive et documenter l'état réseau. Cette réflexe, aussi naturel que le réflexe de sécurisation d'une scène de crime avant toute investigation, s'acquiert par la pratique et la formation.

La troisième leçon est que la super-timeline est l'outil de compréhension le plus puissant disponible à l'analyste. L'investigation forensique produit des quantités considérables de données fragmentées — des milliers d'événements dans les logs, des centaines de milliers d'entrées dans la MFT, des dizaines de processus dans le dump mémoire. Sans un outil permettant de les agréger et de les visualiser chronologiquement, l'analyste ne peut pas reconstituer la narrativité de l'incident. La super-timeline transforme une pile de données brutes en une histoire compréhensible, permettant à l'analyste de répondre aux questions les plus importantes : comment, quand, par qui, et avec quelles conséquences.

Glossaire

Terme	Définition
Acquisition forensique	Processus de copie bit-à-bit d'un support de stockage (disque dur, mémoire RAM, partition) dans le strict respect de l'intégrité des données, accompagné du calcul d'une empreinte cryptographique de vérification.
Artefact forensique	Trace numérique laissée par une activité (légitime ou malveillante) sur un système, exploitable lors d'une investigation : fichiers de log, clés de registre, fichiers prefetch, entrées MFT, données de navigateur.
Chaîne de custody	Documentation rigoureuse traçant la collecte, la conservation et toutes les manipulations d'une preuve numérique, garantissant son intégrité et sa recevabilité légale.
Dump mémoire	Copie complète du contenu de la mémoire vive (RAM) d'un système à un instant donné, préservant les processus actifs, les connexions réseau, les clés de chiffrement et les artefacts d'exécution en mémoire.
Fileless malware	Logiciel malveillant qui s'exécute exclusivement en mémoire vive sans écrire de fichier persistant sur le disque, contournant les antivirus basés sur la détection de fichiers et ne laissant que des traces en RAM.
Hash cryptographique	Empreinte numérique de taille fixe calculée à partir d'un fichier ou d'une donnée via un algorithme (MD5, SHA-1, SHA-256), permettant de vérifier l'intégrité d'une copie forensique.
Image disque (forensique)	Copie bit-à-bit exacte d'un support de stockage incluant les espaces libres, les fichiers supprimés et les métadonnées du système de fichiers, sur laquelle toutes les analyses sont réalisées sans toucher à l'original.
MFT (Master File Table)	Structure centrale du système de fichiers NTFS répertoriant tous les fichiers et répertoires d'un volume avec leurs métadonnées (nom, taille, dates de création/modification/accès, attributs) — y compris les fichiers supprimés.
Ordre de volatilité	Principe forensique imposant de collecter les données dans l'ordre décroissant de leur volatilité (RAM d'abord, disque ensuite), pour préserver les traces les plus éphémères avant qu'elles ne disparaissent.
Patient zéro	Premier système compromis dans une chaîne d'infection, représentant le point d'entrée initial de l'attaquant dans le système d'information — sa détection est un objectif prioritaire de l'investigation forensique.
Prefetch	Mécanisme Windows accélérant le démarrage des applications en préchargeant les ressources nécessaires. Les fichiers prefetch constituent un artefact forensique précieux prouvant l'exécution d'un programme même après sa suppression.
Super-timeline	Chronologie unifiée agrégeant des événements horodatés issus de multiples sources forensiques (logs, MFT, registre, prefetch, navigateurs) permettant de reconstituer la séquence complète d'un incident.
Triage forensique	Phase initiale d'une investigation consistant à identifier rapidement les systèmes compromis et les artefacts les plus pertinents, pour orienter l'investigation approfondie sans analyser l'ensemble des données disponibles.
Write blocker	Dispositif matériel ou logiciel interposé entre l'investigateur et le support à analyser, rendant physiquement impossible toute écriture sur le support original — garant de l'intégrité forensique lors de l'acquisition.

YARA	Langage de description de patterns permettant de créer des règles de détection de malwares basées sur des caractéristiques de leur contenu binaire ou textuel, applicables sur des fichiers, des images disque ou de la mémoire vive.
Zone de temps (timezone)	Référentiel horaire d'un système, critique en forensique : tous les artefacts doivent être normalisés en UTC avant corrélation pour éviter les décalages temporels faussant la reconstruction de la timeline.